

PPGF: Probability Pattern-Guided Time Series Forecasting

Yanru Sun[✉], Zongxia Xie, Haoyu Xing, Hualong Yu, and Qinghua Hu[✉], *Senior Member, IEEE*

Abstract—Time series forecasting (TSF) is an essential branch of machine learning with various applications. Most methods for TSF focus on constructing different networks to extract better information and improve performance. However, practical application data contain different internal mechanisms, resulting in a mixture of multiple patterns. That is, the model's ability to fit different patterns is different and generates different errors. In order to solve this problem, we propose an end-to-end framework, namely probability pattern-guided time series forecasting (PPGF). PPGF reformulates the TSF problem as a forecasting task guided by probabilistic pattern classification. First, we propose the grouping strategy to approach forecasting problems as classification and alleviate the impact of data imbalance on classification. Second, we predict the corresponding class interval to guarantee the consistency of classification and forecasting. In addition, true class probability (TCP) is introduced to pay more attention to the difficult samples to improve the classification accuracy. Detailedly, PPGF classifies the different patterns to determine which one the target value may belong to and estimates it accurately in the corresponding interval. To demonstrate the effectiveness of the proposed framework, we conduct extensive experiments on real-world datasets, and PPGF achieves significant performance improvements over several baseline methods. Furthermore, the effectiveness of TCP and the necessity of consistency between classification and forecasting are proved in the experiments. All data and codes are available online: <https://github.com/syrGit/PPGF>.

Index Terms—Probability pattern classification, simultaneous classification and forecasting, time series forecasting (TSF).

I. INTRODUCTION

TIME series forecasting (TSF), based on observed historical data to predict numerical value fluctuations over time, has attracted growing attention in recent years. It plays a crucial role in many real-world applications, including traffic flow forecasting [1], [2], air quality supervision [3], [4], weather [5], [6], [7], [8], and others [9], [10], [11], [12].

At present, deep neural network (DNN)-based techniques have been developed for TSF tasks [13], [14]. As an essential component of these models, recurrent neural network (RNN)

[15] is an indispensable factor for these noteworthy improvements. Subsequently, the variants of RNN, long short-term memory (LSTM) [16], and gated recurrent unit (GRU) [17] have significantly improved the state-of-the-art performance because they can effectively capture both long- and short-term dependencies. However, RNN-based models can only process one time step at a time, which makes them exceptionally computationally intensive. Gradually, in order to deal with high dimensional data efficiently, convolutional neural network (CNN)-based methods have been developed [18], [19]. Graph neural network (GNN) [20], [21] has been used to capture specific node patterns and automatically infer the dependencies between sequences. In addition, Transformer [22], [23], [24], [25] has been proposed as a new architecture that utilizes attention mechanisms to process sequential data.

The traditional methods consider all the time series samples as a whole and focus on constructing different networks to extract better features to improve performance. However, real-world applications often contain different internal mechanisms, resulting in a mixture of multiple patterns [26], [27]. For example, the slow-speed, steady solar wind caused by coronal holes and the high-speed solar wind caused by coronal mass ejections belong to different patterns due to different generation causes [28]. The direct way is to introduce classification to classify different patterns. As illustrated in Fig. 1, we divide the solar wind speed into five groups to represent different patterns according to their magnitude, and a standard LSTM is trained to predict solar wind speed. We can observe the following: 1) Fig. 1(a): the model has different abilities to fit data in various patterns; 2) Fig. 1(b): the time series data have a long-tailed distribution after grouping; and 3) Fig. 1(c): the model has different performances on different patterns, and patterns with more samples performed better. Therefore, there are three challenges when considering the different patterns in TSF, which can be described as follows:

Challenge 1 (Pattern Grouping): How to define a general pattern grouping method is an important question. Abraham and Tan [29] consider the zeros and the nonzero values in the data. Tan et al. [30] segment the dataset into geomagnetic storm occurrence and nonoccurrence based on the Kp geomagnetic index. Wilson et al. [31] divide the data into extreme and nonextreme events according to a variable threshold U . In addition, DXtreMM [32] sets extreme thresholds for all datasets as second quantile for left extreme and 98th quantile for right extreme. However, most of these methods rely on domain knowledge and cannot be generalized to different

Received 13 June 2023; revised 26 June 2024 and 25 November 2024; accepted 7 February 2025. Date of publication 27 February 2025; date of current version 6 August 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62376194, Grant 61925602, and Grant U23B2049. An earlier version of this paper was presented at the IEEE Publication Technology Group, Piscataway, NJ. (Corresponding author: Zongxia Xie.)

The authors are with the College of Intelligence and Computing, Tianjin Key Laboratory of Machine Learning, Tianjin University, Tianjin 300350, China (e-mail: yanrusun@tju.edu.cn; caddiexie@hotmail.com; hyxing@tju.edu.cn; yhl2000@tju.edu.cn; huqinghua@tju.edu.cn).

Digital Object Identifier 10.1109/TNNLS.2025.3540873

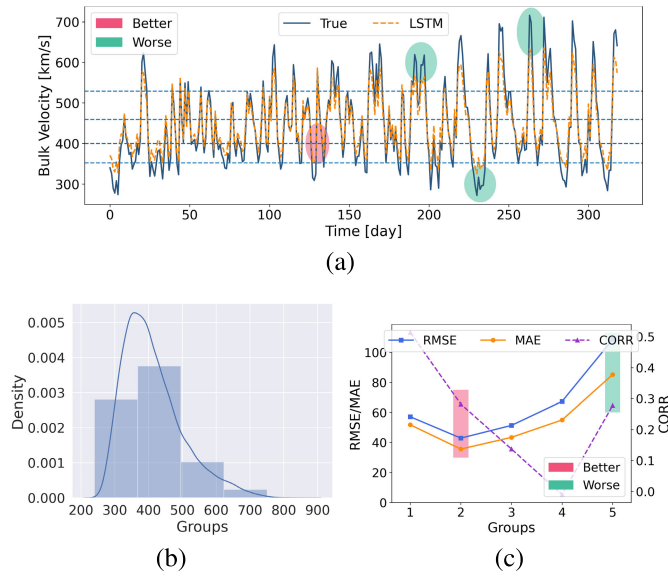


Fig. 1. Relationship between model accuracy and different patterns. We divided the data into five groups representing five patterns. (a) LSTM has different abilities to fit data in various intervals. (b) Time series data are imbalanced and have a long-tailed distribution. (c) Performance of LSTM is disparate for various patterns. The prediction performance is worse for the patterns with few samples, while it is better for the patterns with more samples.

datasets. In addition, most of them do not take into account the effects of data imbalance on classification.

Challenge 2 (Classification Precision): Through pattern grouping, we introduce the classification to mine the patterns in the data, and its accuracy is critical. If the classified pattern is incorrect, the predicted value will be modeled differently, which causes a significant error. Therefore, it is essential to estimate whether a model makes an error and calibrate the classification results.

When we consider introducing the classification into forecasting tasks, it generates another challenge.

Challenge 3 (Consistency of Classification and Forecasting): Some algorithms also consider the multiple patterns contained in time series and decompose TSF into classification and forecasting problems. At first, the two-step approaches are proposed, which train the classifier and the predictors separately [30], [32]. However, these methods require a lot of time and space to train different predictors, and the classifier's performance dramatically affects the model's performance. Recently, some researchers have proposed end-to-end methods that optimize the joint objective function that considers both classification errors and prediction errors [33], [34], [35]. However, these methods have no constraints between classification and forecasting so the prediction and category may belong to a different interval.

To address these challenges, we propose an end-to-end framework, called probability pattern-guided time series forecasting (PPGF). For **pattern grouping**, *Grouping Strategy* is introduced to divide the whole range into multiple groups and avoid the impact of data imbalance on classification. For **classification precision**, we construct a novel structure, *Probability Pattern Classifier*, which introduces true class probability (TCP) [36] to estimate whether a model makes an error

and produce the features' reliability confidence to calibrate the feature. Then the classifier adopts the calibrated feature to achieve the classification task and obtain the category k . For **consistency**, *Relative Prediction Strategy* is designed to produce the relative score ΔY to the lower bound of category k . Then, we propose *Probability Pattern-guided Forecasting* to add the consistency constraint between classification and forecasting, guaranteeing that the prediction value is in the corresponding class interval. The final prediction is obtained according to k and ΔY . PPGF incorporates three loss functions, namely confidence loss ($\mathcal{L}^{\text{conf}}$), cross-entropy loss ($\mathcal{L}^{\text{cls}}$), and mean square error (mse) loss ($\mathcal{L}^{\text{reg}}$), which balances the impact of true label probability, classification, and forecasting.

To summarize, the main contributions of this work are as follows.

- 1) We introduce a PPGF framework that aligns classification and forecasting processes, ensuring that predictions fall within the corresponding classification intervals.
- 2) This article proposes a grouping strategy to mitigate the effects of data imbalance, which does not require expert knowledge and can be flexibly applied to various datasets.
- 3) The proposed PPGF framework uses TCP approximation to estimate the confidence of the classification and calibrate the features accordingly, which can improve pattern classification accuracy effectively.
- 4) Extensive experiments on real-world time series datasets demonstrate that our framework significantly outperforms state-of-the-art methods, while qualitative results validate its effectiveness and interpretability.

II. RELATED WORK

A. Time Series Forecasting

The past few years have witnessed the rapid development of TSF. TSF has many challenges, such as capturing long-term dependencies, noise sensitivity, and computational efficiency [24] and [37]. The prevalent methods in TSF can be categorized into three main groups: Classical methods, Transformer-based methods, and Frequency-domain methods.

1) *Classical Methods*: Bai et al. [38] propose a temporal convolutional network (TCN) architecture combined with CNN, which achieves better performance than RNN while avoiding the common drawbacks of recursive models, such as the gradient explosion/disappearance problem. Dual self-attention network (DSANet) [39] is proposed for multivariable TSF, which uses two parallel convolution components to capture a complex mixture of global and local temporal patterns. Temporal pattern attention (TPA) mechanism is proposed in [40], which extracts the fixed-length temporal patterns in the input information by using CNN filters. TPA determines the weights of each temporal pattern using a scoring function, and obtains the final output according to the magnitude of the weights. Additionally, the multiscale recurrent network (MRN) [41] seamlessly integrates multiscale analysis into deep learning frameworks to build scale-aware recurrent networks. A novel sequence-to-sequence-based deep learning model utilizing a time series decomposition strategy

for multistep load forecasting is proposed in [42]. This model can provide better insights for optimizing energy resource allocation and assisting in the decision-making process. Furthermore, Bai et al. [43] propose adaptive graph convolutional recurrent network (AGCRN) combined with GNN to capture the fine-grained spatiotemporal correlations of traffic sequences automatically.

2) *Transformer-Based Methods*: The emergence of Transformer architectures has revolutionized the ability to capture long-term dependencies in time series data. Li et al. [44] propose the transformer-based architecture to process time series data capturing the long-term dependencies. PatchTST [24] segments the time series into subseries-level patches and adopts the channel independence to improve the long-term forecasting accuracy. iTransformer [25] embeds the time points into variate tokens and applies the feed-forward network for each variate token to empower the Transformer family with promoted performance. However, DLinear [45] adopts a single linear layer to extract the dominant periodicity from the temporal domain and surpasses a range of advanced complex networks.

3) *Frequency-Domain Methods*: On the other hand, many methods have been proposed for TSF from the frequency domain, which are orthogonal to our methods. FITS [46] addresses TSF through interpolation in the complex frequency domain, while FreDF [47] avoids the complexity of label autocorrelation in the time domain by focusing on forecasting in the frequency domain.

Despite the progress in these areas, existing methods often treat time series data as single-pattern data, neglecting the diverse intrinsic mechanisms inherent in time series. This oversight results in varying learning capabilities for distinct patterns, ultimately leading to less accurate predictions. In contrast, we first mine the different patterns of the data and model them by predictor to enhance forecasting performance.

B. Simultaneous Classification and Forecasting

Unlike traditional single tasks, associative tasks can improve performance by sharing information and complementing each other. Specifically, the interaction of unrelated parts between tasks helps to escape local minima. In addition, the relevant parts between tasks facilitate learning generic feature representations at the bottom shared layer.

Combining classification with forecasting has demonstrated a significant potential in TSF. There are two main types. The first type is the two-step method. Tan et al. [30] propose the method to predict the Kp geomagnetic index, which first predicts that the outputs are storm or nonstorm, and then uses two separate submodels to forecast the Kp with and without a geomagnetic storm. DXtreMM [32] improves the extreme events performance by composing a classifier and many forecasting units. The classifier predicts the possibility of occurrence of an extreme event given a time segment, and the forecasting module predicts the exact value. However, these methods consume a lot of time and computing resources due to the training of multiple submodels.

Furthermore, many strategies propose the end-to-end combination of classification and forecasting tasks to improve

extreme events performance. Abraham and Tan [48] present an integrated framework that simultaneously performs classification and forecasting to accurately predict future values of a zero-inflated time series. In addition, DEMM [31] divides the data into three categories according to the magnitude and combines the hurdle model with the extreme value theory to simulate the distribution of different events while accurately making point prediction and distribution prediction.

However, these approaches do not explore the constrained relationship between the two tasks, i.e., even if the classification is correct to class k_i , the predicted result is likely to be class k_j . To address the limitation of the coexistence of two tasks, this article presents a novel approach that introduces a guiding relationship between them.

III. METHOD

A. Problem Description and Grouping Strategy

1) *Problem Description*: Let $A = \{a_1, \dots, a_t\} \in \mathbb{R}^{t \times D}$ be a multivariate time series with channel $D \geq 1$. The data is split into (input, output) pairs using a sliding window approach. Formally, the input matrix of the common TSF method is $X = \{x_i\}_{i=1}^N$ with N samples, where $x_i \in \mathbb{R}^{L \times D}$ and L is the length of the look-back window. The goal of TSF is to predict horizon series: $Y = \{y_i\}_{i=1}^N$, where $y_i \in \mathbb{R}^T$ and T is the length of the horizon.

2) *Grouping Strategy*: A common grouping strategy is the “equal width” strategy, which simply divides the whole interval into multiple equal-width groups. However, this method leads to serious data imbalance in each group, as shown in Fig. 2(a).

To enhance the implementation of pattern grouping, we propose a new strategy that transforms complex temporal relationships into multiple pattern groups. Specifically, we divide A into K nonoverlapping intervals (namely “groups”) to represent different patterns. First, we collect the list of continuous values $A = \{a_1, \dots, a_t\}$, then we arrange them in ascending order to get $\tilde{A} = \{\tilde{a}_1, \dots, \tilde{a}_t\}$. Given the group number K , the partitioning algorithm gives the boundary of each interval $\mathcal{I}_k = (\rho_k^{\text{left}}, \rho_k^{\text{right}})$ as follows:

$$\rho_k^{\text{left}} = \tilde{A} \left(\left\lfloor (t-1) \times \frac{(k-1)}{K} \right\rfloor \right) \quad (1)$$

$$\rho_k^{\text{right}} = \tilde{A} \left(\left\lfloor (t-1) \times \frac{k}{K} \right\rfloor \right) \quad \forall k = 1, 2, \dots, K \quad (2)$$

where t is the length of A and we use $\tilde{A}(k)$ to represent the k th element of $\tilde{A}$.

As shown in Fig. 2(b), the grouping method we propose has two advantages. First, the number of objects in each interval is approximately equal, which alleviates the impact of data imbalance on the results. Moreover, this method is domain-independent and does not rely on expert knowledge.

We can then obtain the target for the relative prediction strategy, denoted as $\Delta Y = \{\Delta y_i\}_{i=1}^N$, which represents the lower bound offset of y with respect to each class k , where $\Delta y_i \in \mathbb{R}^T$. Consequently, the new structure of the dataset is $D = \{x_i, y_i, \Delta y_i, k_i\}_{i=1}^N$, consisting of N samples.

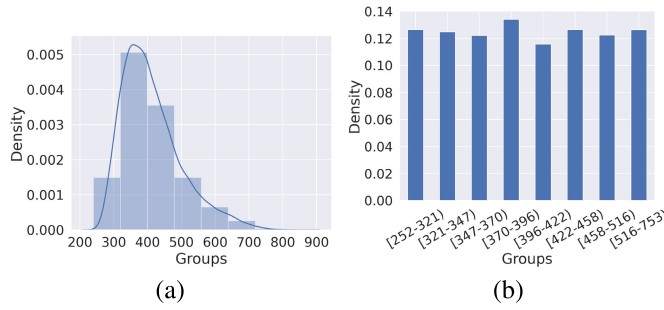


Fig. 2. Distribution of the solar wind in the training set under different partition strategies. (a) Uniform partition. We can observe a large variety of frequencies among different groups. (b) Proposed grouping strategy is given in (2). The data belonging to each group are balanced.

B. Temporal Information Extractor

First, we selected the combination of 1-D convolution, Transformer, and gated residual network (GRN) as the temporal information extractor to capture the general features.

- 1) We adopt the 1-D convolution to capture local dependencies of the input, allowing for rapid feature extraction without extensive computational resources [49]. Its ability to identify localized patterns makes it particularly advantageous in TSF.
- 2) Following the convolutional layer, we employ a Transformer encoder to model long-term dependencies, which are critical for accurate TSF [22]. The combination of convolution and self-attention enhances the model's ability to learn relationships across varying time scales.
- 3) To further refine the model, we incorporate a GRN. Its gating mechanism provides adaptive depth and complexity, allowing the model to dynamically adjust to diverse datasets and scenarios. This adaptability is crucial for handling the heterogeneity often observed in time series data.

By applying the temporal information extractor in the shallow layer of the network, we capture the general temporal information of the time series: $g \in \mathbb{R}^{B \times o}$, where B indicates batch size and o denotes output dimension. Compared to TCNs, which primarily rely on stacked convolutional layers with fixed receptive fields, our approach leverages the flexibility of self-attention in Transformers to dynamically model dependencies across varying time scales. This adaptability, combined with the gating mechanism in GRN, allows our model to better capture both local variations and complex long-term patterns inherent in diverse time series data [38].

C. Probability Pattern Classifier

For time series data, it contains various patterns, and accurate classification is crucial for subsequent relative prediction. Therefore, it is essential for pattern classification to be aware of when an erroneous prediction has been made. In classification tasks, the confidence estimation baseline widely used by neural networks is to take the probability value of the prediction class given by the softmax layer output, namely

the maximum class probability (MCP)

$$\mathcal{L}^{\text{cls}} = - \sum_{k=1}^K k \log p_k \quad (3)$$

where k is the class label. $\mathcal{L}^{\text{cls}}$ is the cross-entropy loss function. However, it is worth mentioning that MCP leads to high confidence values, especially for erroneous predictions making it hard to distinguish errors and correct predictions.

Based on the above motivation, we propose employing TCP [36] to achieve more reliable classification confidence. TCP leverages the softmax output probability associated with the true label as a measure of confidence. When the confidence of a classification is low, it indicates uncertainty, prompting us to potentially adjust the classification result. Formally, given an input time series x , the network produces a probabilistic predictive distribution P for each class, and TCP can be defined as follows:

$$c^* = P(Y = k^* | w, x) \quad (4)$$

where k^* is the real label and w is the parameter of the network. Both TCP and MCP represent the largest softmax outputs for samples that the model correctly classifies, serving as good indicators of classification confidence. However, when the model misclassifies an example, MCP can produce high confidence values for both incorrect and correct predictions, making it challenging to differentiate between them. In contrast, the probability associated with the true label k^* is often closer to a lower value, indicating that the model is likely making an incorrect prediction [36].

Nevertheless, the true label k^* of the output is unavailable when estimating the confidence of the test sample. To address this, we introduce a confidence neural network, termed ConfidNet, with parameters θ . This network outputs a confidence prediction $\hat{c}(h, \theta)$. During training, we optimize θ using the training samples to ensure $\hat{c}(h, \theta)$ approximates $c^*(h, k^*)$. To achieve a regression score between 0 and 1, we train the confidence neural network using ℓ_2 loss

$$\mathcal{L}^{\text{conf}} = (\hat{c}(h, \theta) - c^*(h, k^*))^2. \quad (5)$$

Then the true label probability can be approximated with the classifier and confidence estimation network.

In summary, according to Section III-B, the general feature g can be obtained. In this section, we first adopt a fully connected layer to get the feature vector: $h = \text{FC}_1(g) \in \mathbb{R}^{B \times \text{dim}}$, where B means batch size and dim means the dimension of the hidden layer. Then, a classifier produces the classification probability c , and ConfidNet trained with $\mathcal{L}^{\text{conf}}$ obtains the confidence of the true class, $\hat{c}$. Accordingly, we can assess the reliability of the feature h . A gating strategy is then applied to reweight the feature vector h , allowing the module to obtain calibrated features: $\tilde{h} = h \odot \hat{c}$, where $\odot$ represents elementwise multiplication, as shown in Fig. 4.

Finally, an additional classifier FC_2 is used to obtain the final classification results: $k = \text{FC}_2(\tilde{h})$ according to the calibrated features, and the classifier is trained with cross-entropy loss to minimize the Kullback–Leibler divergence between the

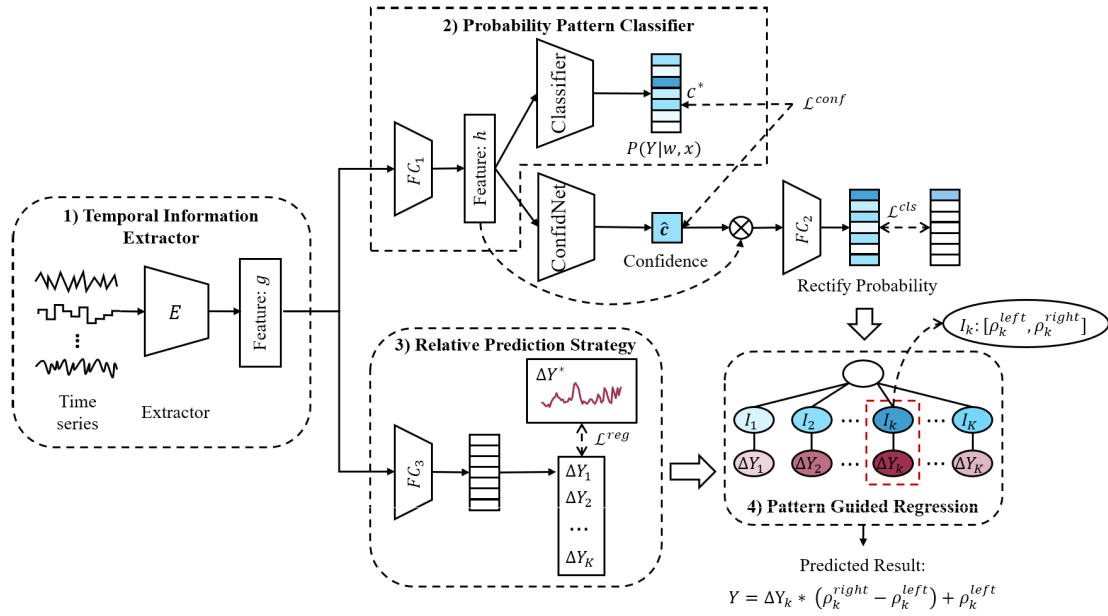


Fig. 3. Pipeline of our proposed probability pattern-guided forecasting method. First, temporal information extractor captures general temporal features at the shallow layer of the network. Then, these features are fed into the probability pattern classifier and relative prediction strategy, respectively, to learn discriminant features and more specific forecasting features. Finally, the probability pattern-guided forecasting strategy is used to get the final prediction results.

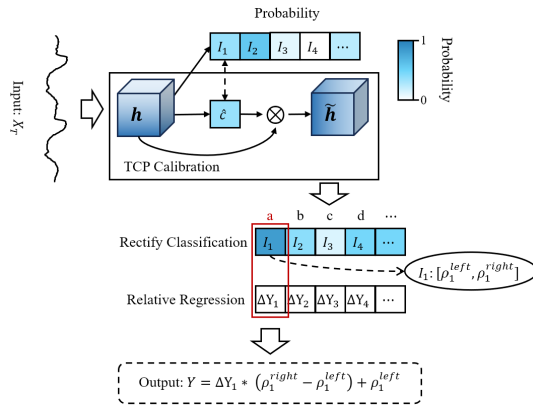


Fig. 4. PPGF strategy. We present the classifier's probability output, the TCP calibration process, and the relative prediction of the predictor.

predictive distribution and the actual distribution

$$\begin{aligned}\mathcal{L}^{\text{cls}} &= \frac{1}{N} \sum_i \mathcal{L}_i \\ &= -\frac{1}{N} \sum_i k_i \log(\omega_i h \odot \hat{c}_i + b)\end{aligned}\quad (6)$$

where ω is the trainable parameter, and b is the bias. We take the derivative of this loss and get

$$\begin{aligned}\frac{\partial \mathcal{L}^{\text{cls}}}{\partial \omega_i} &= -\frac{1}{N} \sum_i k_i \frac{1}{\omega_i h \odot \hat{c}_i + b} \times h \odot \hat{c}_i \\ &= -\frac{1}{N} \sum_i k_i \frac{h}{\omega_i h + \frac{b}{\hat{c}_i}}.\end{aligned}\quad (7)$$

Through (6) and (7), we find that when the classifier makes a mistake, $\hat{c}$ will be small, and the wrong classified sample produces a big loss. When the classification of classifier, c^* , is correct, $\hat{c}$ will be large, and it produces a small loss. Therefore, the model pays more attention to these error samples for better performance.

D. Relative Prediction Strategy

As mentioned in Section III-A, we reformulate the TSF problem as the relative prediction.

We define ΔY as the offset of Y relative to the lower bound of interval k . To predict ΔY , a network $\text{FC}_3 : g \rightarrow \Delta Y$ is trained based on general features obtained by temporal information extractor g

$$\Delta Y = \text{ReLU}(\text{FC}_3(g)) \quad (8)$$

where we denote $\text{FC}_3(g)$ as the full connection layer and adopt the ReLU activation function.

Here, the mse loss is used to ensure the performance of the relative estimation, defined as $\mathcal{L}^{\text{reg}}$

$$\mathcal{L}^{\text{reg}} = (\Delta Y - \hat{\Delta Y})^2 \quad (9)$$

where ΔY means the true label, and $\hat{\Delta Y}$ represents the predicted results.

E. Probability Pattern-Guided Forecasting Strategy

According to Sections III-C and III-D, the classifier performs pattern classification after confidence correcting to determine which pattern the predicted values belong to. Then the regressor makes accurate predictions in the corresponding small interval: ΔY . This section proposes the strategy of probability pattern-guided forecasting to maintain the consistency

between classification and relative forecasting, as shown in Fig. 4.

We perform classification tasks for pattern discovery and apply forecasting tasks to the truth-value pattern interval to train the model. Specifically, when the real predicted value y_i of input sequence x_i belongs to the interval of group k , that is, $\Delta y_k \in \mathcal{I}_k$, and the forecasting label is $\Delta y_k = (y_i - \rho_k^{\text{left}})/(\rho_k^{\text{right}} - \rho_k^{\text{left}})$.

In conclusion, the overall forecasting process of the proposed method can be written as follows:

$$y_i = \Delta y_k * (\rho_k^{\text{right}} - \rho_k^{\text{left}}) + \rho_k^{\text{left}} \quad (10)$$

where k means the group with the highest probability.

F. Objective Function

The total loss function mainly consists of three parts: confidence network loss $\mathcal{L}^{\text{conf}}$, coarse-grained probabilistic pattern classification loss $\mathcal{L}^{\text{cls}}$, and fine-grained relative prediction loss $\mathcal{L}^{\text{reg}}$

$$\mathcal{L} = \sum_{i=1}^N (\lambda_1 \mathcal{L}^{\text{conf}} + \lambda_2 \mathcal{L}^{\text{cls}} + \lambda_3 \mathcal{L}^{\text{reg}}) \quad (11)$$

where λ_1 , λ_2 , and λ_3 are hyperparameters used to balance the influence of true label probability, classification, and forecasting. The model can be obtained by optimizing the loss $\mathcal{L}$. The contribution of these tasks to our current work is different. After experimental verification, this article adopts that $\lambda_1 = \lambda_2 = 1$, and $\lambda_3 = 5$. In a future extension of our model, we will study how to automatically learn weights for each task.

IV. EXPERIMENTS

We conducted extensive experiments with 14 methods on eight available datasets for TSF tasks.

A. Datasets and Analysis

We used the following datasets that are publicly available. Table I summarizes the dataset statistics.

- 1) *Power Consumption (Zone1–3)*¹: The power consumption of Zone 1–3 distribution networks of Tetouan city located in northern Morocco.
- 2) *Steel Industry Energy Consumption*²: Industry energy consumption in kWh collected from a smart small-scale steel industry in South Korea.
- 3) *Metro Interstate Traffic Volume*³: Hourly Minneapolis–St Paul, MN traffic volume for westbound I-94 from 2012 to 2018.
- 4) *Air Quality*⁴ [50]: Recorded by gas multisensor devices deployed on the field in an Italian city.

¹<https://archive.ics.uci.edu/ml/datasets/Power+consumption+of+Tetouan+city#>

²<https://archive.ics.uci.edu/ml/datasets/Steel+Industry+Energy+Consumption+Dataset>

³<https://archive.ics.uci.edu/ml/datasets/Metro+Interstate+Traffic+Volume>

⁴<https://archive.ics.uci.edu/ml/datasets/Air+quality>

TABLE I
DATASET STATISTICS

Datasets	Timesteps	Variates	Granularity
Power consumption (Zone1-3)	52417	7	10 minutes
Steel Industry Energy Consumption	35040	8	15 minutes
Metro Interstate Traffic Volume	40575	3	1 hour
Air Quality	9358	15	1 hour
Electricity Load Diagrams	26304	321	15 minutes
Solar Wind	2545	1	1 day

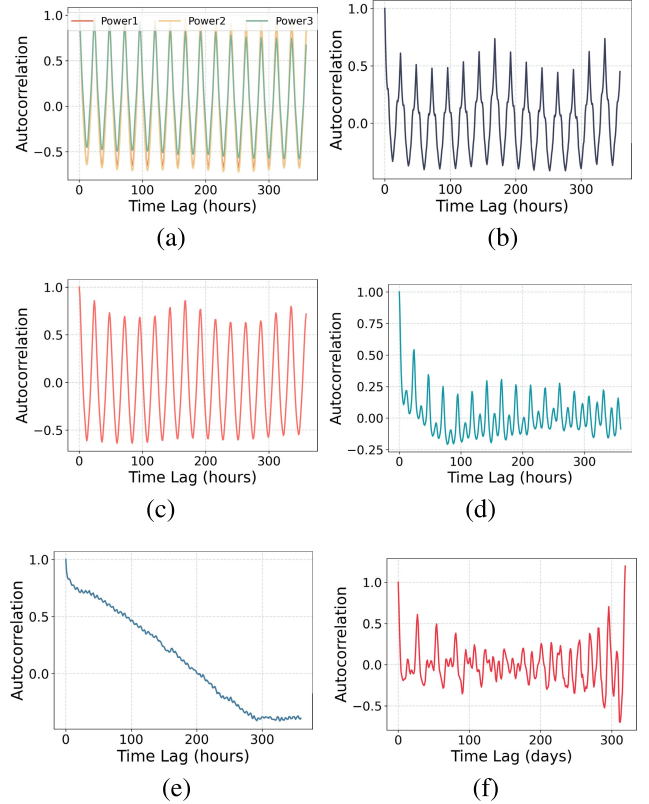


Fig. 5. Autocorrelation graphs of all datasets. (a) Power (Zones 1–3). (b) Steel energy. (c) Traffic volume. (d) Air quality. (e) Electricity load. (f) Solar wind.

5) *Electricity Load Diagrams*⁵ [50]: Electricity consumption (kWh) of 370 clients from 2012 to 2014.

6) *Solar Wind Speed*⁶ [5]: Solar wind speed dataset collected by NASA from 2011 to 2017.

Except the solar wind data, other datasets are divided into training (60%), validation (20%), and testing (20%) in chronological order. Solar wind speed data from 2011 to 2015 are used as the training set, 2016 as the verification set, and 2017 as the test set. In the data preprocessing, the data is normalized.

In order to examine the existence of repetitive patterns and periodicity in time series data, we draw autocorrelation graphs for the target variables of each dataset in Fig. 5. The definition is as follows:

$$R(\tau) = \frac{E(A_t - \mu)(A_{t+\tau} - \mu)}{\sigma^2} \quad (12)$$

⁵<https://archive.ics.uci.edu/ml/datasets/ElectricityLoadDiagrams20112014>

⁶<https://github.com/syrGit/GTA>

where A_t is the time series data, μ is the mean, and σ^2 is the variance.

As seen from graphs (a)–(c) in Fig. 5, there is a highly autocorrelated repeating pattern in the five datasets. However, it is weak in Fig. 5(d)–(f). In addition, we can observe daily and weekly patterns in the graphs of power consumption, steel energy consumption, and traffic volume datasets, which perfectly reflect the regularity in traffic situations and energy consumption. Our experiments have verified that PPGF achieves the best results in all of these datasets. We will revisit this point in Section IV-C.

B. Experimental Setup

1) *Model Comparison and Experimental Setting*: To demonstrate the validity of our approach, we compare it against a diverse set of state-of-the-art forecasting models: 1) traditional TSF models: LSTM [16], TPA [40], MRN [41], AGCRN [43], TCN [38], DSANet [39], Conv-T [44], PatchTST [24], DLinear [45], and Novel Seq2seq [42]; and 2) simultaneous classification and forecasting models: DXterMM [32]. The two-step method indicates that PPGF trains the classifier and the regressor separately, while the multitask strategy allows PPGF to complete classification and forecasting tasks end to end.

2) *Experiments Details*: We conduct a grid search over all hyperparameters on the validation set for each dataset. Specifically, we employ grid search to train PPGF with the learning rate = {0.0001, 0.0005, 0.001, 0.005}, number of pattern = {2, 3, 4, 8}, $\lambda_1 = \{1, 2, 3, 4, 5\}$, $\lambda_2 = \{1, 2, 3, 4, 5\}$, and $\lambda_3 = \{1, 2, 3, 4, 5\}$. Furthermore, we set the batch size to 32 and trained with Adam [51] optimizer. For solar wind dataset, the length of the look-back window L is 27 according to the Sun periodicity, and for other datasets, the length of the look-back window is 32. To ensure the fairness of the experiment, we reran DXtreMM, PatchTST, and DLinear according to original papers and reproduced the other deep learning models according to the hyperparameter settings provided by Sun et al. [50].

3) *Evaluation Metrics*: We use six evaluation metrics defined as follows.

- 1) *Classification*: For the classification task, we mainly adopt Precision, Recall, and F1 score to evaluate the performance of models

$$\text{Macro-Precision} = \frac{\sum_{i=1}^K \text{Pre}(i)}{K} \quad (13)$$

$$\text{Macro-Recall} = \frac{\sum_{i=1}^K R(i)}{K} \quad (14)$$

$$\begin{aligned} \text{Macro-F1} &= \frac{1}{K} \sum_{i=1}^K F1(i) \\ &= \frac{1}{K} \sum_{i=1}^K \frac{2 * \text{Pre}(i) * R(i)}{\text{Pre}(i) + R(i)} \end{aligned} \quad (15)$$

where Pre and R mean Precision and Recall, respectively. Higher is better for all three metrics.

- 2) *Forecasting*: For the forecasting task, we mainly use RMSE, MAE, and CORR to evaluate the model's overall

performance

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - y_i^*)^2} \quad (16)$$

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |y_i - y_i^*| \quad (17)$$

$$\text{CORR} = \frac{\sum_{i=1}^N (y_i^* - \bar{y}_i^*)(y_i - \bar{y}_i)}{\sqrt{\sum_{i=1}^N (y_i^* - \bar{y}_i^*)^2} \sqrt{\sum_{i=1}^N (y_i - \bar{y}_i)^2}} \quad (18)$$

where y_i^* is the i th predicted speed value, y_i is the i th observed speed value, $\bar{y}_i^*$ and $\bar{y}_i$ are the mean value of the predicted and observed speed value respectively, and N is the number of samples. For RMSE and MAE, the lower value is better, but for CORR, the higher value is better.

C. Main Results

Table II highlights the consistent superiority of PPGF across multiple datasets, securing the top performance in 19 of 24 experimental configurations. In particular, PPGF reduces the RMSE by about 107% compared to the baselines for power consumption, steel industry energy consumption, and traffic volume—datasets characterized by significant periodicity. For datasets with weak periodic patterns, such as air quality, electricity load, and solar wind, PPGF still demonstrates robust performance, achieving RMSE reductions of 39%, 61%, and 30%, respectively. These results underscore the effectiveness of our model, which we attribute to three primary design aspects. First, the Pattern-Guided Forecasting Strategy simplifies TSF by introducing a grouping strategy that discretizes continuous labels into bins and forecasts values within the corresponding intervals. This approach enables PPGF to capture variations more effectively, even in the absence of pronounced periodic patterns. Second, the probability pattern classifier provides more reliable classification information, and when combined with the relative prediction strategy, it improves performance across diverse datasets. Finally, PPGF's architecture is designed to leverage both local and global dependencies in the data, allowing it to adapt to different patterns and complexities inherent in time series data. This flexibility is particularly advantageous for datasets where traditional models may struggle to identify and exploit underlying trends.

What is more, by comparing with DXtreMM and two-step PPGF, we can conclude that end to end is a better strategy because it considers the classification and forecasting errors jointly. And DXtreMM achieves better performance than the traditional TSF models, proving that it is essential to consider different patterns in the data. Second, PPGF obtains the best performance compared with the multitask strategy, which should be attributed to the classification-guided forecasting strategy. We alleviate the data imbalanced problem and limit the target value to a small interval through the pattern classification task.

In addition, we find that DXtreMM outperforms PPGF on some datasets, especially those with large magnitudes, such as

TABLE II
PERFORMANCE COMPARISON WITH BENCHMARK MODELS. THE BEST RESULTS ARE HIGHLIGHTED IN BOLD

Datasets/Models		Metrics	Traditional Time Series Forecasting										Simultaneous Classification and Forecasting			
			LSTM	TPA	MRN	AGCRN	TCN	DSANet	Conv-T	PatchTST	DLinear	Novel Seq2seq	DXterMM	Two-Step	Multi-Task	PPGF
Power Consumption	Zone 1	RMSE	1063.08	4833.56	855.93	2257.54	881.72	924.25	1250.79	845.80	971.75	813.90	528.28	992.96	4829.22	790.18
		MAE	864.79	3943.29	630.51	1757.11	489.56	682.45	943.78	640.18	624.09	543.44	382.07	700.97	2640.23	532.56
		CORR	0.98	0.73	0.98	0.93	0.99	0.98	0.98	0.99	0.99	0.99	0.99	0.99	0.93	0.99
	Zone 2	RMSE	939.21	895.43	541.05	2997.82	750.27	860.23	626.79	501.93	539.32	600.26	514.64	831.11	5633.07	439.77
		MAE	731.81	677.85	436.43	2509.59	549.62	649.74	464.88	333.19	324.14	450.28	318.84	543.03	2744.52	317.48
		CORR	0.98	0.98	0.98	0.92	0.98	0.98	0.99	0.99	0.99	0.99	1.00	0.99	0.90	1.00
	Zone 3	RMSE	1545.45	898.96	834.63	2327.14	1900.30	1029.88	2315.99	784.21	819.61	820.40	708.76	944.87	3563.93	699.02
		MAE	1096.43	728.67	636.36	1715.92	1428.51	762.55	1902.19	584.83	549.12	586.66	362.95	498.66	1783.89	441.18
		CORR	0.94	0.97	0.94	0.78	0.89	0.97	0.82	0.97	0.97	0.97	0.98	0.96	0.88	0.98
Steel Industry Energy Consumption	RMSE	11.49	11.73	11.54	25.15	12.67	12.16	25.56	11.44	11.71	11.94	17.36	10.11	47.84	9.81	
	MAE	6.28	7.66	6.46	6.71	6.71	7.75	16.69	5.72	6.50	6.24	8.24	6.59	27.28	5.33	
	CORR	0.93	0.93	0.93	0.91	0.91	0.93	0.59	0.93	0.93	0.93	0.84	0.95	0.86	0.96	
Metro Interstate Traffic Volume	RMSE	682.56	1145.31	485.03	544.50	858.50	863.93	542.33	450.75	450.93	486.94	425.80	650.66	1067.66	396.27	
	MAE	541.45	809.34	395.34	680.57	680.57	664.07	418.73	316.81	326.79	361.79	280.33	501.87	858.69	282.04	
	CORR	0.94	0.90	0.95	0.92	0.92	0.93	0.98	0.97	0.96	0.97	0.98	0.96	0.92	0.98	
Air Quality	RMSE	0.93	0.90	0.86	1.32	0.97	1.21	1.36	0.77	0.77	0.86	0.97	0.84	0.98	0.71	
	MAE	0.64	0.64	0.59	1.01	0.67	0.89	1.05	0.53	0.55	0.61	0.64	0.50	0.78	0.48	
	CORR	0.74	0.75	0.77	0.24	0.76	0.66	0.61	0.83	0.82	0.78	0.78	0.80	0.79	0.86	
Electricity Load Diagrams	RMSE	24.42	15.13	15.35	23.62	10.12	10.24	24.91	9.74	9.79	11.88	13.63	16.88	12.67	9.49	
	MAE	19.20	9.81	8.23	20.73	6.13	6.97	22.35	4.86	5.30	7.44	6.36	10.41	9.86	4.22	
	CORR	0.85	0.81	0.84	0.34	0.92	0.93	0.84	0.92	0.91	0.89	0.85	0.84	0.93	0.92	
Solar Wind Speed	RMSE	97.51	86.11	84.23	89.81	78.79	87.88	101.77	80.36	78.80	98.87	84.79	76.16	129.85	69.51	
	MAE	79.92	67.62	71.04	73.73	59.54	66.63	84.78	63.41	60.99	76.54	62.40	64.31	107.22	52.49	
	CORR	0.62	0.55	0.58	0.49	0.69	0.65	0.51	0.56	0.66	0.69	0.66	0.75	0.64	0.77	
1 st			0	0	0	0	1	1	1	1	1	8	1	0	19	

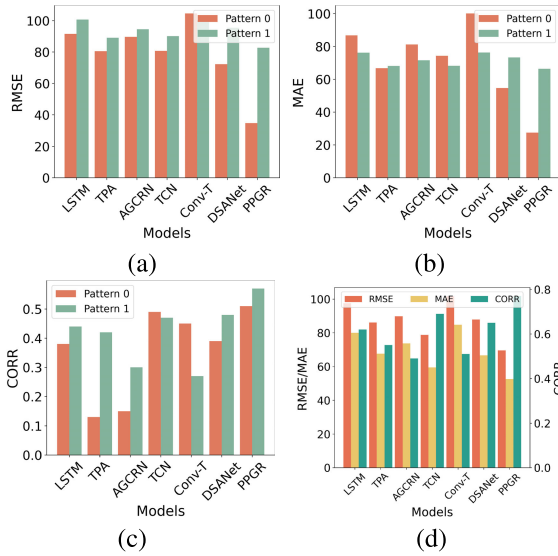


Fig. 6. (a) RMSE, (b) MAE, (c) CORR performance on different patterns, and (d) overall results on the solar wind dataset.

power consumption datasets. The reason is that DXtreMM is specifically designed for extreme event prediction, and extreme values are more common in these datasets.

D. Classification Pattern

This section aims to demonstrate the performance obtained for two patterns in the solar wind dataset, as well as to assess the impact of the number of patterns on the performance of various datasets. Specifically, we present the performance on the solar wind dataset under $K = 2$, and investigate the effect of changing pattern numbers across different datasets.

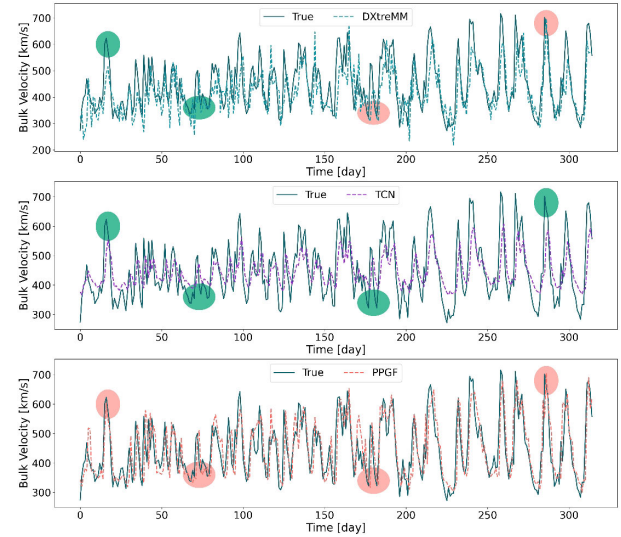


Fig. 7. Visualizations of three models on the solar wind dataset. Compared with other models, PPGF fits the data of different patterns well.

1) *Results of Different Patterns:* Coronal holes or coronal mass ejections typically cause high-speed solar wind, while low-speed solar wind generally represents the background solar wind from the Sun's surface. The two types of solar wind belong to different patterns. In Fig. 6, pattern 0 means the low-speed solar wind, and pattern 1 represents the high-speed solar wind. We can find that by introducing classification task to model different modes in different intervals, PPGF is helpful in improving the performance of each pattern.

Furthermore, we visualize the prediction results of the three models on the solar wind dataset in Fig. 7. It can be seen that compared with other models, PPGF alleviates the

TABLE III

RMSE OF THE DIFFERENT NUMBER OF PATTERN IN THE CLASSIFICATION INFORMATION ESTIMATOR MODULE. THE BEST RESULTS ARE HIGHLIGHTED IN BOLD

Number of Pattern		2	3	4	8	Standard Deviation
Power Consumption	Zone 1	889.47	873.42	877.69	790.17	7173.43
	Zone 2	695.11	787.04	575.38	439.77	4970.96
	Zone 3	798.63	962.92	699.01	936.18	6464.89
Steel Industry Energy Consumption		10.2	9.81	9.84	12.69	33.92
Metro Interstate Traffic Volume		458.02	453.15	396.27	494.16	1988.40
Air Quality		0.70	0.83	0.72	0.72	1.58
Electricity Load Diagrams		9.49	28.23	26.47	10.01	24.04
Solar Wind		69.51	69.84	71.43	78.64	88.53

problems in Fig. 1 to a large extent and achieves a good fit for different patterns. We contribute these improvements to introducing the classification task, which narrows the interval of the forecasting task, reduces the difficulty of prediction, and achieves the best performance. Because DXtreMM is specially designed for extreme values prediction, it performs well at some peaks. The rest of the methods do not consider the category information or differentiate between different patterns, resulting in poor fit ability.

2) *Interpretation of the Number of Pattern K*: Through experiments, we find that the number of patterns has an essential effect on the experimental results. Table III shows the results of different patterns for the forecasting task.

We find that better results are achieved for Power Consumption Zone 1 and Zone 2 datasets when $K = 8$, with RMSE of 790.17 and 439.77, respectively. For the Zone 3 and Traffic Volume datasets, the best results are achieved when $K = 4$, with an RMSE of 699.01 and 396.27, respectively. For the Steel Industry Energy Consumption, better results are obtained when $K = 3$, with RMSE of 9.81. For other datasets, we get the best performance with $K = 2$.

Furthermore, we analyze the relationship among the data distributions, the pattern number, and the performance. As shown in Table III, we adopt the standard deviation of each dataset to represent the degree of imbalance. We find that when the data distribution is more imbalanced, the larger the class number K is, the better effect is achieved. We can conclude that more detailed grouping can better alleviate the imbalance phenomenon.

E. Ablation Experiments

To demonstrate the effectiveness of our framework, ablation studies are conducted. Specifically, we remove one component of PPGF at a time. First, we name the PPGF without different components as follows.

- 1) *PPGFw/oC*: The PPGF model directly forecasting without the classification.
- 2) *PPGFw/oR*: The PPGF model without the Relative Prediction Strategy.
- 3) *PPGFw/oConv1d*: The PPGF model without the 1-D convolution component of Temporal Information Extractor.

TABLE IV

ABLATION RESULTS OF RMSE, MAE, AND CORR ON SOLAR WIND DATASET. THE BEST RESULTS ARE HIGHLIGHTED IN BOLD

	Forecasting			MACRO-Classification		
	RMSE	MAE	CORR	Precision	Recall	F1
PPGFw/oR	--	--	--	77.82%	77.62%	77.72
PPGFw/oC	78.12	62.07	0.71	--	--	--
PPGFw/oConv1d	81.55	60.25	0.67	76.37%	76.09%	76.23
PPGFw/oTransformer	84.74	64.59	0.61	70.31%	69.81%	70.06
PPGFw/oGRN	85.34	67.65	0.64	74.23%	73.74%	73.98
PPGFw/oGroup	74.72	54.95	0.71	80.32%	78.65%	79.48
PPGFw/oConf	96.92	70.81	0.58	76.83%	76.24%	76.53
PPGF	69.51	52.49	0.77	82.18%	82.17%	82.17

- 4) *PPGFw/oTransformer*: The PPGF model without the Transformer component of Temporal Information Extractor.
- 5) *PPGFw/oGRN*: The PPGF model without the GRN component of Temporal Information Extractor.
- 6) *PPGFw/oGroup*: The PPGF model adopting the “equal width” strategy for grouping.
- 7) *PPGFw/oConf*: The PPGF model directly classifies without the calibration of probability pattern classifier.

Test results measured with three forecasting metrics and three classification metrics are shown in Table IV. A few observations from these results are worth highlighting.

- 1) The best result on each metric is obtained with PPGF.
- 2) Removing either the classifier component (PPGFw/oC) or the predictor component (PPGFw/oR) from the full model leads to a significant decrease in performance, demonstrating both components’ critical role and proving that the interrelationship between the two tasks plays an essential role in performance improvement.
- 3) After removing Conv1d (PPGFw/oConv1d), Transformer (PPGFw/oTransformer), or GRN (PPGFw/oGRN), RMSE increases. This indicates that these components help PPGF better capture dependencies between multivariate input features and temporal dynamics, demonstrating their rationality and effectiveness.
- 4) When we adopt the equal width strategy to group the samples, the classification precision obviously decreases, which results in the poor predicted performance.
- 5) The removal of the ConfidNet module (PPGFw/oConf) leads to a degradation in the performance of the classification task, which in turn affects the prediction performance. The results show that the probability pattern classifier adjusts the classification features according to reliability. On the other hand, it also demonstrates that the patterns do play a guiding role in the forecasting.

The conclusion is that our architectural design is the most robust among all experimental setups.

F. Case Study

- 1) *Model Calibration Process*: To better understand the model’s behavior, we present a case study in Fig. 8.

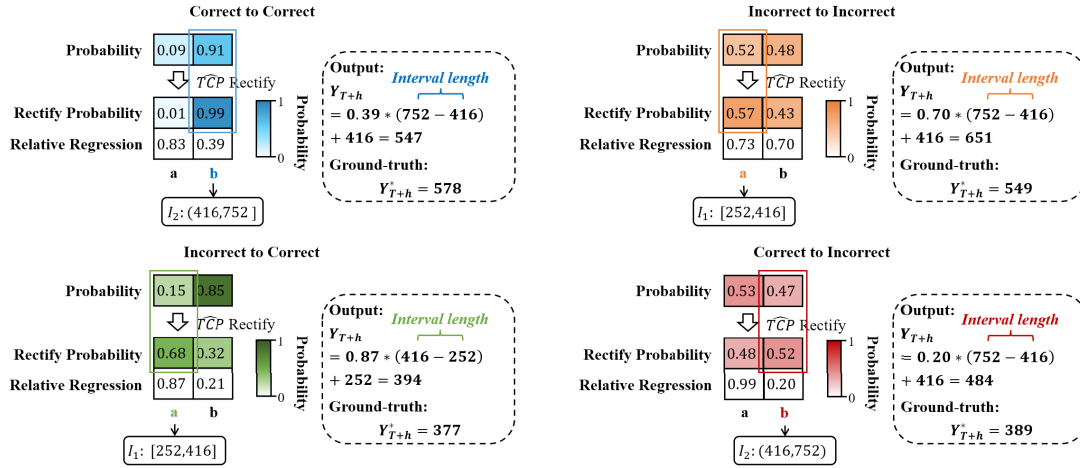


Fig. 8. Case study. We show the probability output of the pattern classifier T , the final classifier, and the relative prediction of the PPGF strategy. We take the prediction of the pattern with the highest probability as the final forecasting result. When calibrated successfully, the error between the predicted results and the targets is minor, proving the calibration mechanism's importance.

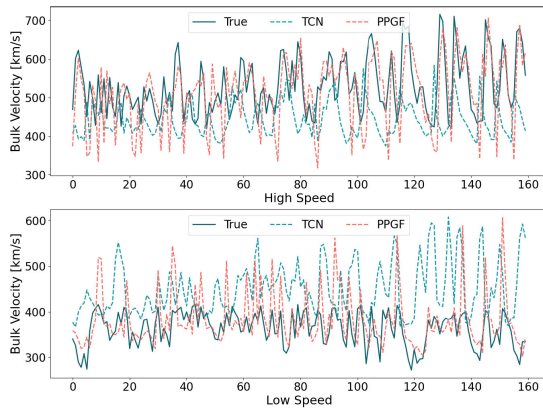


Fig. 9. Predicted results of PPGF and TCN according to different patterns on solar wind dataset. TCN predicts Y directly. PPGF predicts ΔY in a compressed interval to improve the performance.

The probability pattern-guided forecasting strategy performs coarse-to-fine hierarchical forecasting based on the input samples. The probability pattern classifier first determines which pattern the target value belongs to, and the relative prediction strategy attempts to make the prediction more accurate. The four cases in Fig. 8 show the result on solar wind dataset when the number of pattern $K = 2$. The first case shows the situation of correct classification, and the second demonstrates that when the pattern classification is wrong, it is not correct after the modification. The third case represents the samples that are misclassified but successfully calibrated. Moreover, the fourth shows the example classified correctly but falsely calibrated. From the four situations, we find that when the pattern is misclassified, the prediction will be in the wrong interval, resulting in a large error. In contrast, when the pattern is correct, it will produce a small error, proving the importance of calibration mechanism.

2) *Role of Pattern-Guided Forecasting*: We draw the predicted results of PPGF and TCN according to different patterns, shown in Fig. 9. We find that PPGF achieves accurate performance in the parts of samples that are easy to classify correctly. As stated in Section III-E, the predicted Y with

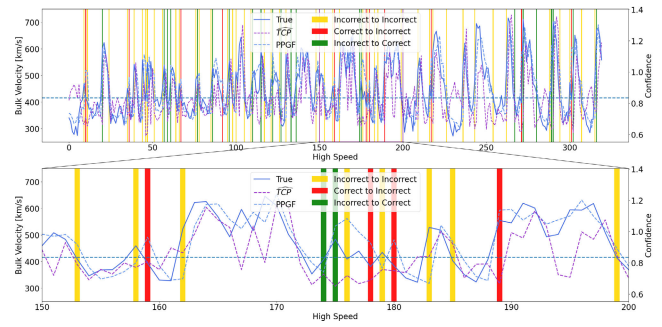


Fig. 10. Predicted values, true values, and $\hat{c}$ on solar wind dataset. The yellow bars indicate that the samples are misclassified and are not calibrated. The red bars indicate that the samples are classified correctly and falsely calibrated. The green bars indicate that the samples are misclassified and successfully calibrated. The rest represents samples originally classified correctly and still classified correctly after correction.

PPGF depends on the category classified by the probability pattern classifier and the offset ΔY . We conducted the mse loss on ΔY , not on the predicted Y . Therefore, PPGF predicts ΔY in a compressed range compared with TCN that predicts Y directly, which is helpful for forecasting.

In addition, we plot $\hat{c}$ in Fig. 10 to visualize the function of probability pattern classifier. The yellow bars indicate that the samples are misclassified and are not calibrated. The red bars indicate that the samples are classified correctly but falsely calibrated. The green bars indicate that the samples are misclassified but successfully calibrated. The rest represents samples originally classified correctly and still classified correctly after correction. From Fig. 10, we find that our proposed calibration mechanism is effective and successfully calibrate some misclassified samples correctly. Although some samples are calibrated incorrectly, overall, the number of samples correctly calibrated by this method is higher, which is consistent with Table IV. In addition, we find that when the mistake occurs, $\hat{c}$ is low. As mentioned in Section III-C, when the classifier makes a mistake, $\hat{c}$ is small, which leads the model to pay more attention to these misclassification samples to improve performance.

V. CONCLUSION

In this article, we propose a probability pattern-guided time series forecasting framework to achieve more accurate prediction. Considering the mixture of multiple patterns implicit in the data, we design a flexible pattern grouping strategy, which transforms the traditional prediction problem into pattern classification and interval relative forecasting tasks with constraints. Moreover, we introduce TCP approximation to realize the reliable pattern recognition. Experiments on real-world datasets demonstrate that the proposed approach significantly improves the state-of-the-art results in TSF on multiple benchmark datasets. Through in-depth analysis and empirical evidence, we demonstrate the PPGF model architecture's efficiency, and that it indeed successfully achieves robust prediction through reliable pattern classification.

Although the proposed method can solve the imbalance in forecasting to a certain degree, the pattern number is taken as a hyperparameter which is chosen according to the results of experiments. The pattern number should be determined based on the imbalance degree of the dataset, which is our future work to realize the data-driven way in the proposed method. What is more, PPGF is prone to misclassifying samples near the boundary, affecting the performance. We will pay attention as to how to improve the prediction accuracy of these samples in the future. In addition, the temporal information extractor can use other advanced networks to enhance its effectiveness and applicability.

REFERENCES

- [1] Q. Long, Z. Fang, C. Fang, C. Chen, P. Wang, and Y. Zhou, "Unveiling delay effects in traffic forecasting: A perspective from spatial-temporal delay differential equations," in *Proc. ACM Web Conf.*, May 2024, pp. 1035–1044.
- [2] W. Kong, Z. Guo, and Y. Liu, "Spatio-temporal pivotal graph neural networks for traffic flow forecasting," in *Proc. AAAI Conf. Artif. Intell.*, 2024, vol. 38, no. 8, pp. 8627–8635.
- [3] Z. B. Patel, P. Purohit, H. Patel, S. Sahni, and N. Batra, "Accurate and scalable Gaussian processes for fine-grained air quality inference," in *Proc. AAAI Conf. Artif. Intell.*, Jun. 2022, vol. 36, no. 11, pp. 12080–12088.
- [4] C. Yu et al., "MGSFformer: A multi-granularity spatiotemporal fusion transformer for air quality prediction," *Inf. Fusion*, vol. 113, Jan. 2025, Art. no. 102607.
- [5] Y. Sun, Z. Xie, H. Wang, X. Huang, and Q. Hu, "Solar wind speed prediction via graph attention network," *Space Weather*, vol. 20, no. 7, Jul. 2022, Art. no. e2022SW003128.
- [6] K. Bi, L. Xie, H. Zhang, X. Chen, X. Gu, and Q. Tian, "Accurate medium-range global weather forecasting with 3D neural networks," *Nature*, vol. 619, no. 7970, pp. 533–538, Jul. 2023.
- [7] H. Wu, H. Zhou, M. Long, and J. Wang, "Interpretable weather forecasting for worldwide stations with a unified deep model," *Nature Mach. Intell.*, vol. 5, no. 6, pp. 602–611, Jun. 2023.
- [8] R. Lam et al., "Learning skillful medium-range global weather forecasting," *Science*, vol. 382, no. 6677, pp. 1416–1421, Dec. 2023.
- [9] M. N. Akhter et al., "An hour-ahead PV power forecasting method based on an RNN-LSTM model for three different PV plants," *Energies*, vol. 15, no. 6, p. 2243, Mar. 2022.
- [10] J. Wang, Y. Zhou, and Z. Li, "Hour-ahead photovoltaic generation forecasting method based on machine learning and multi objective optimization algorithm," *Appl. Energy*, vol. 312, Apr. 2022, Art. no. 118725.
- [11] H. Wang et al., "An accurate and interpretable framework for trustworthy process monitoring," *IEEE Trans. Artif. Intell.*, vol. 5, no. 5, pp. 2241–2252, May 2024.
- [12] Z. Liu, G. Zheng, and Y. Yu, "Cross-city few-shot traffic forecasting via traffic pattern bank," in *Proc. 32nd ACM Int. Conf. Inf. Knowl. Manage.*, Oct. 2023, pp. 1451–1460.
- [13] X. Liu and W. Wang, "Deep time series forecasting models: A comprehensive survey," *Mathematics*, vol. 12, no. 10, p. 1504, May 2024.
- [14] X. Qiu et al., "TFB: Towards comprehensive and fair benchmarking of time series forecasting methods," *Proc. VLDB Endowment*, vol. 17, no. 9, pp. 2363–2377, May 2024.
- [15] J. L. Elman, "Finding structure in time," *Cogn. Sci.*, vol. 14, no. 2, pp. 179–211, Mar. 1990.
- [16] A. Graves, "Long short-term memory," in *Supervised Sequence Labelling with Recurrent Neural Networks*. Berlin, Germany: Springer, 2012, pp. 37–45.
- [17] J. Chung, C. Gulcehre, K. Cho, and Y. Bengio, "Empirical evaluation of gated recurrent neural networks on sequence modeling," 2014, *arXiv:1412.3555*.
- [18] H. Wu, T. Hu, Y. Liu, H. Zhou, J. Wang, and M. Long, "Timesnet: Temporal 2D-variation modeling for general time series analysis," in *Proc. 11th Int. Conf. Learn. Represent.*, 2023, pp. 143–165.
- [19] D. Luo and X. Wang, "ModernTCN: A modern pure convolution structure for general time series analysis," in *Proc. 12th Int. Conf. Learn. Represent.*, 2024, pp. 432–474.
- [20] Z. Shao, Z. Zhang, F. Wang, and Y. Xu, "Pre-training enhanced spatial-temporal graph neural network for multivariate time series forecasting," in *Proc. 28th ACM SIGKDD Conf. Knowl. Discovery Data Mining*, Aug. 2022, pp. 1567–1577.
- [21] F. Li et al., "Dynamic graph convolutional recurrent network for traffic prediction: Benchmark and solution," *ACM Trans. Knowl. Discovery Data*, vol. 17, no. 1, pp. 1–21, Feb. 2023.
- [22] H. Zhou et al., "Informer: Beyond efficient transformer for long sequence time-series forecasting," in *Proc. AAAI Conf. Artif. Intell.*, 2021, vol. 35, no. 12, pp. 11106–11115.
- [23] H. Wu, J. Xu, J. Wang, and M. Long, "Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting," in *Proc. NIPS*, vol. 34, Dec. 2021, pp. 22419–22430.
- [24] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam, "A time series is worth 64 words: Long-term forecasting with transformers," in *Proc. 11th Int. Conf. Learn. Represent.*, 2022, pp. 649–672.
- [25] Y. Liu et al., "ITransformer: Inverted transformers are effective for time series forecasting," in *Proc. 12th Int. Conf. Learn. Represent.*, Jan. 2023, pp. 323–341. [Online]. Available: <https://openreview.net/forum?id=JePfAI8fah>
- [26] Y. Li, J. Xu, and D. C. Anastasiu, "An extreme-adaptive time series prediction model based on probability-enhanced LSTM neural networks," 2022, *arXiv:2211.15891*.
- [27] Z. Liu et al., "Adaptive normalization for non-stationary time series forecasting: A temporal slice perspective," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 36, 2023, pp. 1–11.
- [28] Y. Sun, Z. Xie, Y. Chen, and Q. Hu, "Accurate solar wind speed prediction with multimodality information," *Space, Sci. Technol.*, vol. 2022, Jan. 2022, pp. 89–101.
- [29] Z. Abraham and P.-N. Tan, "A semi-supervised framework for simultaneous classification and regression of zero-inflated time series data with application to precipitation prediction," in *Proc. IEEE Int. Conf. Data Mining Workshops*, Dec. 2009, pp. 644–649.
- [30] Y. Tan, Q. Hu, Z. Wang, and Q. Zhong, "Geomagnetic index kp forecasting with LSTM," *Space Weather*, vol. 16, no. 4, pp. 406–416, Apr. 2018.
- [31] T. Wilson, A. McDonald, A. H. Galib, P.-N. Tan, and L. Luo, "Beyond point prediction: Capturing zero-inflated & heavy-tailed spatiotemporal data with deep extreme mixture models," in *Proc. 28th ACM SIGKDD Conf. Knowl. Discovery Data Mining*, Aug. 2022, pp. 2020–2028.
- [32] S. Abilasha, S. Bhadra, A. Z. Dadarkar, and P. Deepak, "Deep extreme mixture model for time series forecasting," in *Proc. 31st ACM Int. Conf. Inf. Knowl. Manage.*, Oct. 2022, pp. 1726–1735.
- [33] X. Yu, Y. Rao, W. Zhao, J. Lu, and J. Zhou, "Group-aware contrastive regression for action quality assessment," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, Jul. 2021, pp. 7919–7928.
- [34] X. Jiang, L. Meng, and D. Wu, "Multi-task active learning for simultaneous emotion classification and regression," in *Proc. IEEE Int. Conf. Syst., Man, Cybern. (SMC)*, Oct. 2021, pp. 1947–1952.
- [35] M. M. Ben Ismail and N. AlSadhan, "Simultaneous classification and regression for zakat under-reporting detection," *Appl. Sci.*, vol. 13, no. 9, p. 5244, Apr. 2023.
- [36] C. Corbière, N. Thome, A. Bar-Hen, M. Cord, and P. Pérez, "Addressing failure prediction by learning model confidence," in *Proc. Adv. Neural Inf. Process. Syst.*, Jan. 2019, pp. 1–12.
- [37] E. Eldele, M. Ragab, Z. Chen, M. Wu, and X. Li, "TSLANet: Rethinking transformers for time series representation learning," in *Proc. 41st Int. Conf. Mach. Learn.*, Apr. 2024, pp. 523–541.

- [38] S. Bai, J. Zico Kolter, and V. Koltun, "An empirical evaluation of generic convolutional and recurrent networks for sequence modeling," 2018, *arXiv:1803.01271*.
- [39] S. Huang, D. Wang, X. Wu, and A. Tang, "DSANet: Dual self-attention network for multivariate time series forecasting," in *Proc. 28th ACM Int. Conf. Inf. Knowl. Manage.*, Nov. 2019, pp. 2129–2132.
- [40] S.-Y. Shih, F.-K. Sun, and H.-Y. Lee, "Temporal pattern attention for multivariate time series forecasting," *Mach. Learn.*, vol. 108, nos. 8–9, pp. 1421–1441, Sep. 2019.
- [41] Q. Guo, L. Fang, R. Wang, and C. Zhang, "Multivariate time series forecasting using multiscale recurrent networks with scale attention and cross-scale guidance," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 36, no. 1, pp. 540–554, Jan. 2023.
- [42] R. Lu et al., "A novel sequence-to-sequence-based deep learning model for multistep load forecasting," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 36, no. 1, pp. 638–652, Jan. 2025.
- [43] L. Bai, L. Yao, C. Li, and X. Wang, "Adaptive graph convolutional recurrent network for traffic forecasting," in *Proc. NIPS*, 2020, pp. 17804–17815.
- [44] S. Li et al., "Enhancing the locality and breaking the memory bottleneck of transformer on time series forecasting," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 32, Jan. 2019, pp. 1–12.
- [45] A. Zeng, M. Chen, L. Zhang, and Q. Xu, "Are transformers effective for time series forecasting?" in *Proc. AAAI Conf. Artif. Intell.*, Jun. 2023, vol. 37, no. 9, pp. 11121–11128.
- [46] Z. Xu, A. Zeng, and Q. Xu, "Fits: Modeling time series with 10k parameters," in *Proc. The 12th Int. Conf. Learn. Represent.*, 2023, pp. 8973–8996.
- [47] H. Wang et al., "FreDF: Learning to forecast in frequency domain," 2024, *arXiv:2402.02399*.
- [48] Z. Abraham and P.-N. Tan, "An integrated framework for simultaneous classification and regression of time-series data," in *Proc. SIAM Int. Conf. Data Mining*, Apr. 2010, pp. 653–664.
- [49] A. Dempster, F. Petitjean, and G. I. Webb, "ROCKET: Exceptionally fast and accurate time series classification using random convolutional kernels," *Data Mining Knowl. Discovery*, vol. 34, no. 5, pp. 1454–1495, Sep. 2020.
- [50] F.-K. Sun, C. I. Lang, and D. S. Boning, "Adjusting for autocorrelated errors in neural networks for time series," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 34, Jan. 2021, pp. 29806–29819.
- [51] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. Int. Conf. Learn. Represent.*, Dec. 2015, pp. 2234–2248.



Yanru Sun received the B.S. degree in computer science and technology from Shanxi University, Shanxi, China, 2015, and the M.S. degree in computer technology major from Tianjin University, Tianjin, China, 2019, where she is currently pursuing the Ph.D. degree in computer science and technology with the College of Intelligence and Computing.

Her research interests include deep learning, time series forecasting, and graph neural networks.



Zongxia Xie received the B.S. degree from Dalian Maritime University, Dalian, China, in 2003, and the M.S. and Ph.D. degrees from Harbin Institute of Technology, Harbin, China, in 2005 and 2010, respectively.

From December 2010 to January 2013, she was a Post-Doctoral Researcher at Shenzhen Graduate School, Harbin Institute of Technology. She is currently an Associate Professor at the College of Intelligence and Computing, Tianjin University, Tianjin, China. Her major interests include machine learning and pattern recognition.



Haoyu Xing received the B.S. degree in computer science from the College of Intelligence and Computing, Tianjin University, Tianjin, China, in 2022, where he is currently pursuing the M.S. degree.

His research interests include deep learning, time series forecasting, and representation learning.



Hualong Yu received the B.S. degree in software engineering from Zhengzhou University, Zhengzhou, Henan, China, in 2018. He is currently pursuing the M.S. degree with the College of Intelligence and Computing, Tianjin University, Tianjin, China.

His research interests include deep learning and time series forecasting.



Qinghua Hu (Senior Member, IEEE) received the B.S., M.S., and Ph.D. degrees from Harbin Institute of Technology, Harbin, China, in 1999, 2002, and 2008, respectively.

After that he joined the Department of Computing, The Hong Kong Polytechnic University, Hong Kong, as a Post-Doctoral Fellow. He became a Full Professor with Tianjin University, Tianjin, China, in 2012, and currently the Chair Professor and the Deputy Dean of the College of Intelligence and Computing. He has published more than 300 peer-reviewed

articles in IEEE TRANSACTIONS ON KNOWLEDGE AND DATA ENGINEERING, IEEE TRANSACTIONS ON PATTERN ANALYSIS AND MACHINE INTELLIGENCE, and IEEE TRANSACTIONS ON NEURAL NETWORKS AND LEARNING SYSTEMS. His research interests include uncertainty modeling, multimodality learning, incremental learning, and continual learning these years, funded by the National Natural Science Foundation of China and The National Key Research and Development Program of China.

Dr. Hu was a recipient of the Best Paper Award of ICMLC 2015 and ICME 2021. He is an Associate Editor of IEEE TRANSACTIONS ON FUZZY SYSTEMS, *Acta Automatica Sinica*, and *Acta Electronica Sinica*.